Meditation State Classifier

Pipeline Technical Documentation

Training Script I Testing Script I Signal Processing I Model Method

1. Overview

This document describes the end-to-end pipeline of two Python scripts: **train_meditation_classifier.py** and **test_meditation_classifier.py**. Together they train a supervised classifier on a 64-channel benchmark EEG dataset from Thai Buddhist monks and then apply it to real-time recordings from a Muse 2 headband.

The pipeline consists of three phases:

- Offline feature engineering on the benchmark dataset
- Supervised model training with cross-validation
- Real-time inference with quality gating on Muse 2 data

1.1 Target Meditation Levels

Four discrete states are classified, derived from the unsupervised clustering phase conducted earlier on the monk dataset:

Level	Name	Dominant Band	Description
L0	Baseline	13-45 Hz (Beta/Gamma)	Active, distracted state
L1	Relaxed	8-13 Hz (Alpha)	Physical calm, eyes closed
L2	Focused	4-8 Hz (Theta)	Sustained meditative focus
L3	Deep	1-4 Hz (Delta)	Profound stillness (expert-level)

1.2 EEG Frequency Band Definitions

Band	Range	Associated State
Delta (δ)	`1-4 Hz`	Deep sleep, profound stillness
Theta (θ)	`4-8 Hz`	Meditation, drowsiness, creative states
Alpha (α)	`8-13 Hz`	Relaxed wakefulness, eyes-closed calm

Band	Range	Associated State
Beta (β)	13-30 Hz	Active thinking, concentration, alertness
Gamma (γ)	30-45 Hz	High cognitive load, arousal

2. Training Script

File: `train_meditation_classifier.py`

The training script reads the labelled benchmark dataset, engineers 19 features per row, normalises them, and fits a Random Forest classifier. It saves the model, scaler, and feature name list as pickle files for use during testing.

2.1 Input Data

- File: meditation_levels_output.csv
- ~342 rows, one per 5-minute EEG window
- 64 EEG channels with absolute and relative band power columns
- Column naming: CH_Band (absolute) and CH_Band/all (relative, 0-1 normalised)
- Target column: meditation_level (integer 0-3, from prior clustering)

2.2 Feature Engineering

19 features are extracted from the 64-channel monk data using the same structure that can be reproduced from only 4 Muse 2 channels at test time. This ensures training and inference are compatible.

Grp	Feature Name	Description	Count
A	`mean_rel_Delta/Theta/Alpha/Beta/Gamma`	Mean relative power across all channels per band	5
B	`std_rel_Delta/Theta/Alpha/Beta`	Std deviation across channels per band	4
C	`ratio_theta_alpha, ratio_alpha_beta, ratio_delta_alpha, ratio_theta_beta, ratio_delta_beta, ratio_gamma_alpha`	Pairwise spectral ratios	6
D	`frontal_alpha_asymm frontal_theta_mean occipital_alpha_mean`	Regional spatial aggregates	3
		TOTAL	**19**

Group A — Mean Relative Band Power

For each of the five frequency bands, we compute the mean relative (normalised) power across all channels:

```
mean_rel_Band = (1/N) * sum( P_rel(ch, Band) )  for ch in all channels

where:  P_rel(ch, Band) = P_abs(ch, Band) / sum_over_all_bands( P_abs(ch, b) )
        N = number of channels with that band column
```

The relative power is already provided in the dataset's /all columns (e.g. AF3_Delta/all). When missing, it is computed from absolute power columns by dividing by the total power across all bands.

Group B — Spatial Standard Deviation

The standard deviation of relative power across channels measures how unevenly a band is distributed across the scalp. Meditation tends to produce more uniform, widespread alpha/theta.

```
std_rel_Band = std({ P_rel(ch1,Band), P_rel(ch2,Band), ... })

Computed for: Delta, Theta, Alpha, Beta  (4 features)
```

Group C — Spectral Ratios

Ratios between bands are more robust to inter-subject amplitude differences than absolute powers. A small constant epsilon prevents division by zero.

```
epsilon = 1e-8

ratio_theta_alpha  = mean_rel_Theta  / (mean_rel_Alpha + epsilon)
ratio_alpha_beta   = mean_rel_Alpha  / (mean_rel_Beta  + epsilon)
ratio_delta_alpha  = mean_rel_Delta  / (mean_rel_Alpha + epsilon)
ratio_theta_beta   = mean_rel_Theta  / (mean_rel_Beta  + epsilon)
ratio_delta_beta   = mean_rel_Delta  / (mean_rel_Beta  + epsilon)
ratio_gamma_alpha  = mean_rel_Gamma  / (mean_rel_Alpha + epsilon)
```

Group D — Regional / Spatial Features

These capture the geography of EEG activity, which carries additional information about meditation depth beyond global averages.

```
// Frontal alpha asymmetry (linked to emotional regulation and focus)
```

```
frontal_alpha_asymm = mean(AF3,F3,F7 Alpha_rel) - mean(AF4,F4,F8 Alpha_rel)

// Frontal theta (strongest meditation indicator in prefrontal cortex)
frontal_theta_mean = mean(AF3, AF4, Fz, FCz Theta_rel)

// Occipital alpha (eyes-closed relaxation marker)
occipital_alpha_mean = mean(O1, O2, Oz, PO3, PO4 Alpha_rel)
```

2.3 Feature Normalisation (StandardScaler)

Before model training, all 19 features are scaled to zero mean and unit variance. This prevents features with larger numerical ranges from dominating split decisions.

```
X_scaled = (X - mu) / sigma

mu    = mean of each feature across all training rows
sigma = standard deviation of each feature across all training rows

The fitted scaler (mu and sigma) is saved to: scaler.pkl
It is reused at test time so the same transformation is applied.
```

2.4 Model — Random Forest Classifier

A Random Forest is an ensemble of decision trees. Each tree is trained on a bootstrap sample (sampling with replacement) of the training data, and at each split considers only a random subset of features. This reduces overfitting and variance.

Gini Impurity — Split Criterion

Each internal node in a decision tree is split by finding the feature and threshold that minimises Gini impurity, which measures class mixing at a node:

```
Gini(t) = 1 - sum_c [ p(c|t)^2 ]

p(c|t) = fraction of samples at node t belonging to class c
Gini = 0 means pure node (all same class)
Gini = 0.75 means maximum mixing (4 equal classes)
```

The best split is chosen by maximising the Gini gain (weighted reduction in impurity):

```
Gini_gain = Gini(parent) - [ (|L|/N)*Gini(L) + (|R|/N)*Gini(R) ]
```

```
|L|, |R| = number of samples in left and right child nodes
N         = total samples at the parent node
```

Ensemble Prediction

The final class prediction is the majority vote across all 500 trees. Probability estimates are the fraction of trees voting for each class:

```
y_hat = argmax_c [ sum_{k=1}^{500} indicator(tree_k predicts c) ]

P(c | x) = (1/500) * count of trees predicting class c for input x

confidence = max( P(c|x) )   // used as per-window quality score
```

Hyperparameters

- n_estimators = 500 trees
- max_depth = None (trees grow until leaves are pure or min_samples_leaf is reached)
- min_samples_leaf = 2 (prevents single-sample leaves)
- class_weight = 'balanced' (up-weights minority classes by N_total / (N_classes * N_class))

2.5 Cross-Validation Strategy

To measure generalisation to unseen subjects, Subject-Independent GroupKFold is used. All windows from a given monk subject are assigned to only one fold, so the model is always tested on subjects it never saw during training.

```
n_splits = min(5, n_unique_subjects)

For each fold k:
  Training set = all windows from subjects NOT in fold k
  Test set     = all windows from subjects IN fold k

CV_accuracy = mean accuracy across all k folds
CV_std      = standard deviation across folds

Fallback: if fewer than 5 subjects -> StratifiedKFold(5, shuffle=True)
```

2.6 Training Outputs

- meditation_rf_classifier.pkl — serialised trained Random Forest model
- model_feature_names.pkl — ordered list of the 19 feature names
- scaler.pkl — fitted StandardScaler (mu and sigma per feature)

- confusion_matrix.png — heatmap of predicted vs actual labels on training data
- feature_importance.png — Gini-based mean decrease impurity per feature
- training_report.txt — CV scores, classification report, per-class precision/recall/F1

3. Testing Script

File: `test_meditation_classifier.py`

Loads a Muse 2 CSV recording and produces a per-window meditation level prediction. The Muse 2 exports pre-computed band powers directly (Delta_TP9, Alpha_AF7, etc.) so no raw FFT is performed here. The script windows those values, engineers the same 19 features, and classifies.

3.1 Input Data — Muse 2 CSV Format

The Muse 2 headband (4 channels: TP9, AF7, AF8, TP10) records at 256 Hz and exports a CSV with the following column families:

- Delta_TP9, Theta_TP9, Alpha_TP9, Beta_TP9, Gamma_TP9 (and same for AF7, AF8, TP10)
- - These are relative band powers (0.0-1.0 scale), already normalised by the Muse firmware
- RAW_TP9, RAW_AF7, RAW_AF8, RAW_TP10 — raw microvolts (not used for features here)
- Accelerometer_X/Y/Z — motion data in g-force units
- HeadBandOn — 1 if headband is correctly worn, 0 otherwise
- HSI_TP9, HSI_AF7, HSI_AF8, HSI_TP10 — contact quality (1=Good, 2=OK, 3=Poor, 4=VeryPoor)
- TimeStamp — ISO datetime string, e.g. 2025-11-14 11:35:38.265

3.2 Step 1 — Load and Parse

The script auto-detects band-power columns by matching Band_Channel patterns, parses ISO timestamps, and estimates the actual sample rate from inter-sample time differences:

```
delta_t = diff(timestamps[:500])                    // array of time gaps in
seconds
FS_estimated = round( 1 / median(delta_t) )         // robust to dropped samples

// Accepted range: 50 to 512 Hz
// Defaults to 256 Hz if estimation fails
```

3.3 Step 2 — Signal Quality Gating

Before windowing, rows with poor electrode contact are flagged using the Muse's built-in quality indicators. Windows with too many bad samples are discarded rather than producing unreliable predictions.

```
quality_ok[i] = (HeadBandOn[i] == 1) AND (all HSI columns[i] <= 2)

// A window is SKIPPED if:  mean(quality_ok in window) < 0.70
// i.e., more than 30% of samples in the window have poor contact
```

3.4 Step 3 — Motion Artefact Gating (IMU)

Head movement contaminates EEG signals. The accelerometer resultant magnitude is computed, the DC component (gravity) removed, and windows with high motion variability are excluded:

```
resultant[i]    = sqrt( Ax[i]^2 + Ay[i]^2 + Az[i]^2 )   // units: g
r_detrended[i]  = resultant[i] - mean(resultant)           // remove ~1g
gravity
motion_std      = std(r_detrended)                         // per window

// Window is SKIPPED if:  motion_std > 0.08 g
// Threshold tunable via MOTION_THRESHOLD constant
```

3.5 Step 4 — Windowing

Valid rows are segmented into overlapping windows. Within each window the band-power columns are averaged across time, giving one representative value per band per channel:

```
WINDOW_SAMP = FS * 2          // e.g. 512 samples at 256 Hz (2 seconds)
STRIDE_SAMP = FS * 1          // 256 samples -> new window every 1 second

n_windows = floor( (N_rows - WINDOW_SAMP) / STRIDE_SAMP ) + 1

// 50% overlap: each window shares half its samples with the next.
// This produces smoother classification transitions.

per_window_band[w, Band, ch] = mean( Band_ch[w_start : w_end] )
```

3.6 Step 5 — Feature Extraction (same 19 features as training)

For each valid window, the 19 features are computed from the 4 Muse channels, exactly mirroring the training feature definitions:

```
// Channels available: TP9, AF7, AF8, TP10

mean_rel_Band[w]   = mean( Band_TP9[w], Band_AF7[w], Band_AF8[w], Band_TP10[w]
)
std_rel_Band[w]    = std ( Band_TP9[w], Band_AF7[w], Band_AF8[w], Band_TP10[w]
)

// Spectral ratios (identical formulas to training)
ratio_theta_alpha  = mean_rel_Theta  / (mean_rel_Alpha + 1e-8)
ratio_alpha_beta   = mean_rel_Alpha  / (mean_rel_Beta  + 1e-8)
ratio_delta_alpha  = mean_rel_Delta  / (mean_rel_Alpha + 1e-8)
ratio_theta_beta   = mean_rel_Theta  / (mean_rel_Beta  + 1e-8)
ratio_delta_beta   = mean_rel_Delta  / (mean_rel_Beta  + 1e-8)
ratio_gamma_alpha  = mean_rel_Gamma  / (mean_rel_Alpha + 1e-8)

// Regional features mapped to closest Muse 2 channels:
frontal_alpha_asymm  = Alpha_AF7[w]  - Alpha_AF8[w]
frontal_theta_mean   = mean( Theta_AF7[w], Theta_AF8[w] )
occipital_alpha_mean = mean( Alpha_TP9[w], Alpha_TP10[w] )
   // TP9/TP10 are temporal-occipital — closest Muse channels to occipital lobe
```

3.7 Step 6 — Scaling and Prediction

The 19 feature values are scaled using the saved scaler (same mu and sigma fitted during training), then passed to the Random Forest for classification:

```
X_scaled[w] = ( X[w] - mu_train ) / sigma_train     // loaded from scaler.pkl

y_hat[w]    = RF.predict( X_scaled[w] )              // integer 0, 1, 2, or 3

P[w]        = RF.predict_proba( X_scaled[w] )        // [P_L0, P_L1, P_L2,
P_L3]
confidence  = max( P[w] )                            // highest class
probability
```

3.8 Note on Signal Processing

Why no Welch PSD or band-pass filter in testing?

The Muse 2 firmware performs its own internal DSP and exports relative band powers directly in the CSV (Delta_TP9, etc.). These are equivalent to what an external Welch PSD pipeline would produce from raw EEG. Applying additional band-pass filtering to already-processed band power values would be incorrect double-processing.

The RAW_TP9/AF7/AF8/TP10 columns (raw microvolts) are available if you need to implement a full custom DSP chain (band-pass -> notch -> Welch PSD). For the current pipeline they are not used for feature extraction.

3.9 Testing Outputs

- meditation_predictions.csv — per-window predictions, probabilities, and feature values
- meditation_timeline.png — 3-panel time-series: predicted level / confidence / band powers

4. End-to-End Workflow

```
// Step 1: Run training (once)
python train_meditation_classifier.py

// Produces:
//   meditation_rf_classifier.pkl
//   model_feature_names.pkl
//   scaler.pkl

// Step 2: Copy the 3 pkl files to the testing directory
cp meditation_rf_classifier.pkl  /Users/syian/Desktop/testttt/
cp model_feature_names.pkl       /Users/syian/Desktop/testttt/
cp scaler.pkl                    /Users/syian/Desktop/testttt/

// Step 3: Run inference on a Muse 2 recording
python test_meditation_classifier.py

// Produces:
//   meditation_predictions.csv
//   meditation_timeline.png
```

5. Dependencies

- pandas — data loading and manipulation
- numpy — numerical operations
- scikit-learn — StandardScaler, RandomForestClassifier, GroupKFold, metrics
- matplotlib — plot generation
- scipy — signal processing utilities (used if extending to raw EEG)
- pickle — model serialisation

```
pip install pandas numpy scikit-learn matplotlib scipy
```